第17课 模拟法（一）

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：翻转数字（入门级）

题目：

输入一个正整数 n（1 ≤ n ≤ 99999），输出它翻转后的结果。例如输入 `1230`，输出 `321`（去掉前导零）。

讲解思路——翻译三步法演示：

> "读剧本：把一个数字倒过来。
> 列步骤：怎么把 `123` 倒过来呢？想象你有一堆积木，每次从右边拿一个放到新的一排——
> 1. 从 n 的末尾取出一位数字（用 `n % 10`）
> 2. 把这个数字放到结果的末尾（`result = result * 10 + 这个数字`）
> 3. 把 n 的末尾砍掉（`n = n / 10`）
> 4. 重复，直到 n 变成 0"

变量跟踪表（板书演示 n = 1230）：

步骤	n	digit	result
初始	1230	—	0
第1轮	123	0	0
第2轮	12	3	3
第3轮	1	2	32
第4轮	0	1	321

> "看！前导零自动就没了，因为 0×10+3 就是 3，不会有前面的 0。"

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int result = 0;        // 记分板：翻转后的结果
    while (n > 0) {        // 只要剧本没演完（n还有数字）
        int digit = n % 10;            // 第一步：取出末尾数字
        result = result * 10 + digit;  // 第二步：放到结果末尾
        n = n / 10;                    // 第三步：砍掉末尾
    }

    cout << result << endl;
    return 0;
}
```

师生互动点：
- 请一个同学上来，在纸上当"人肉电脑"，手动跟踪 n = 520 的执行过程。
- 提问："如果输入是 100，结果是多少？"（答：1。提醒学生注意前导零的处理是自动的。）

例题 2：时钟进位（进阶级）

题目：

一个电子时钟显示 HH:MM:SS 格式的时间（用 24 小时制）。给定当前时间的时 h、分 m、秒 s，再给定经过的秒数 t（0 ≤ t ≤ 100000），请输出 t 秒后时钟显示的时间（格式 HH:MM:SS，不足两位补前导零）。

输入示例： `23 59 45 20`
输出示例： `00:00:05`

讲解思路：

> "想象你家墙上的时钟，秒针一格一格走。每走一格：
> - 秒数 +1
> - 如果秒到了 60？归零！分钟 +1
> - 如果分钟到了 60？归零！小时 +1
> - 如果小时到了 24？归零！（新的一天）
>
> 这就是进位！和我们十进制加法的进位是一样的道理。"

方法一：直接一秒一秒模拟（最直观，但 t 大时慢）

> "最笨但最好理解的办法：每次让秒 +1，然后检查要不要进位。循环 t 次。"

方法二：数学转换（推荐，更快）

> "聪明的办法：把时间全部变成秒数，加上 t，再变回时分秒。就像把人民币全换成'分'来算，最后再换回'元角分'。"

完整代码（方法二）：

```
#include <iostream>
#include <iomanip>   // 用来补前导零
using namespace std;

int main() {
    int h, m, s, t;
    cin >> h >> m >> s >> t;

    // 第一步：把现在的时间全部变成秒数（统一单位）
    int total = h * 3600 + m * 60 + s;

    // 第二步：加上经过的秒数
    total = total + t;

    // 第三步：处理超过一天的情况（24小时 = 86400秒）
    total = total % 86400;

    // 第四步：把总秒数变回时、分、秒
    h = total / 3600;            // 有多少个小时
    m = (total % 3600) / 60;     // 剩下的有多少个分钟
    s = total % 60;              // 最后剩下多少秒

    // 第五步：按格式输出（不足两位补0）
    cout << setfill('0') << setw(2) << h << ":"
         << setw(2) << m << ":"
         << setw(2) << s << endl;

    return 0;
}
```

补充讲解 `setfill` 和 `setw`：

> "`setw(2)` 是说'这个数字至少占 2 格'，`setfill('0')` 是说'如果不够就用 0 来填'。就像学校运动会号码牌，01 号、02 号，前面要补零。"

师生互动点：
- 在黑板上手动演算 23:59:45 + 20秒 的过程。
- 提问："如果 total 超过了 86400（一天的秒数），为什么要取余？"（答：时钟不会显示 25 点，到 24 就归零重新来。）

例题 3：简单日期推算（挑战级）

题目：

输入年份 y 和一个天数 d（1 ≤ d ≤ 365），输出这一年的第 d 天是几月几日。不考虑闰年，即假设每年都是 365 天，2 月都是 28 天。

输入示例： `2024 60`
输出示例： `3 1`（即 3 月 1 日）

讲解思路：

> "想象你在撕日历。1 月有 31 页，你一页页撕。如果你要撕到第 60 页：
> - 先撕 1 月的 31 页，还剩 60-31=29 页要撕
> - 再撕 2 月的 28 页，还剩 29-28=1 页
> - 到 3 月了，只需要撕 1 页 答案是 3 月 1 日！
>
> 这个'一个月一个月减'的过程，就是模拟！"

列步骤：
1. 用一个数组存每个月有多少天。
2. 从 1 月开始，看 d 够不够减去这个月的天数。
3. 如果够，就减掉，进入下个月。
4. 如果不够，说明就在这个月，d 就是几号。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int y, d;
    cin >> y >> d;

    // 每个月的天数（不考虑闰年）
    // 下标0不用，1~12分别代表1月~12月
    int days[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    int month = 1;  // 从1月开始撕日历

    while (d > days[month]) {   // 如果剩余天数比这个月的总天数多
        d = d - days[month];    // 撕掉这个月，剩余天数减少
        month++;                // 进入下个月
    }

    // 循环结束时，d就是当月的第几天
    cout << month << " " << d << endl;

    return 0;
}
```

变量跟踪表（d = 60）：

步骤	month	d	days[month]	d > days[month]?
初始	1	60	31	60 > 31
第1轮后	2	29	28	29 > 28
第2轮后	3	1	31	1 > 31 停！

> "答案就是 3 月 1 日！是不是和我们撕日历的过程一模一样？"

师生互动点：
- 让学生用变量跟踪表手动模拟 d = 32（答案 2 月 1 日）和 d = 365（答案 12 月 31 日）。
- 提问："如果 d = 31，答案是几月几号？"（答：1 月 31 日。因为 31 不大于 31，循环不会执行。）

附：例题源代码与运行结果

例题 1：Reverse a Number

```
// Input a positive integer n (1 <= n <= 99999), output its reversed form.
// Leading zeros are automatically removed.

#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int result = 0;       // Stores the reversed number
    while (n > 0) {       // Keep going until no digits left
        int digit = n % 10;           // Step 1: extract the last digit
        result = result * 10 + digit;  // Step 2: append digit to result
        n = n / 10;                    // Step 3: remove the last digit
    }

    cout << result << endl;
    return 0;
}
```

Sample Input

```
1230
```

Expected Output

```
321
```

例题 2：Clock Time Calculation

```
// Given current time (h, m, s) and elapsed seconds t,
// compute the time after t seconds in HH:MM:SS format (24-hour, zero-padded).

/*
#include <iostream>
#include <iomanip>   // For setfill and setw to pad zeros
using namespace std;

int main() {
    int h, m, s, t;
    cin >> h >> m >> s >> t;

    // Step 1: Convert current time to total seconds
    int total = h * 3600 + m * 60 + s;

    // Step 2: Add elapsed seconds
    total = total + t;

    // Step 3: Handle overflow past 24 hours (86400 seconds per day)
    total = total % 86400;

    // Step 4: Convert total seconds back to hours, minutes, seconds
    h = total / 3600;
    m = (total % 3600) / 60;
    s = total % 60;

    // Step 5: Output in HH:MM:SS format with leading zeros
    cout << setfill('0') << setw(2) << h << ":"
         << setw(2) << m << ":"
         << setw(2) << s << endl;

    return 0;
}
*/
```

Sample Input

```
23 59 45 20
```

Expected Output

```
00:00:05
```

例题 3：Day of Year to Month and Day

```
// Given a year y and day number d (1 <= d <= 365),
// output the month and day (no leap year handling).

/*
#include <iostream>
using namespace std;

int main() {
    int y, d;
    cin >> y >> d;

    // Days in each month (index 0 unused, 1-12 for Jan-Dec, no leap year)
    int days[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    int month = 1;  // Start from January

    while (d > days[month]) {   // If remaining days exceed this month's total
        d = d - days[month];    // Subtract this month's days
        month++;                // Move to next month
    }

    // When loop ends, d is the day within the current month
    cout << month << " " << d << endl;

    return 0;
}
*/
```

Sample Input

```
2024 60
```

Expected Output

```
3 1
```